

# CS 374: Section A - Homework 5, Problem 1

Anirudh Konidala

October 6, 2026

Interval  $I_i = [a_i, b_i]$  is given by  $A[i] = a_i$  and  $B[i] = b_i$ , all  $2n$  values are distinct, and

$$\text{overlap}(i, j) = \max\{0, \min\{b_i, b_j\} - \max\{a_i, a_j\}\}.$$

We want the largest value of  $\text{overlap}(i, j)$  over all pairs  $i \neq j$ , and we assume  $n \geq 2$  so that at least one such pair exists. The algorithm only compares input values and subtracts them; it uses no hashing and no radix sort.

## Algorithm

**Preprocessing:** Begin by sorting the intervals by left endpoint with mergesort, permuting  $B$  alongside  $A$  so that  $A[i]$  and  $B[i]$  still describe the same interval. After this step  $A[1] < A[2] < \dots < A[n]$ . This takes  $O(n \log n)$  time, and it does not change the answer, since the answer does not depend on how the intervals are numbered.

**Main recursion:**  $\text{MAXOVERLAP}(A[1..n], B[1..n])$ , given  $n$  intervals sorted by left endpoint, returns the maximum overlap length between any two distinct intervals among them, or 0 if  $n < 2$ .

```
MAXOVERLAP( $A[1..n], B[1..n]$ ):  
  if  $n < 2$   
    return 0    ⟨⟨no pair of distinct intervals⟩⟩  
   $m \leftarrow \lfloor n/2 \rfloor$   
   $left \leftarrow \text{MAXOVERLAP}(A[1..m], B[1..m])$   
   $right \leftarrow \text{MAXOVERLAP}(A[m+1..n], B[m+1..n])$   
   $cross \leftarrow \text{CROSSOVERLAP}(A[1..n], B[1..n], m)$   
  return  $\max\{left, right, cross\}$ 
```

**Conquer step.**  $\text{CROSSOVERLAP}(A[1..n], B[1..n], m)$ , given  $n$  intervals sorted by left endpoint and a split index  $m$ , returns the maximum of  $\text{overlap}(i, j)$  over all pairs with  $i \leq m < j$ , that is, one interval from the left half and one from the right half.

```

CROSSOVERLAP( $A[1..n], B[1..n], m$ ):
     $bmax \leftarrow B[1]$     ⟨largest right endpoint in left half⟩
    for  $i \leftarrow 2$  to  $m$ 
         $bmax \leftarrow \max\{bmax, B[i]\}$ 
     $best \leftarrow 0$ 
    for  $j \leftarrow m + 1$  to  $n$ 
         $best \leftarrow \max\{best, \min\{bmax, B[j]\} - A[j]\}$ 
    return  $best$ 

```

The answer to the problem is  $\text{MAXOVERLAP}(A[1..n], B[1..n])$  called on the sorted arrays.

**Example.** For  $A = \langle 1, 7, 4, 5 \rangle$  and  $B = \langle 8, 10, 9, 6 \rangle$ , sorting gives the intervals  $[1, 8]$ ,  $[4, 9]$ ,  $[5, 6]$ ,  $[7, 10]$ . The left half returns  $\min\{8, 9\} - 4 = 4$ , the right half returns  $\max\{0, \min\{6, 10\} - 7\} = 0$ , and the cross step has  $bmax = 9$  and returns  $\max\{\min\{9, 6\} - 5, \min\{9, 10\} - 7\} = 2$ . The answer is 4, as required.

## Why this is correct

Every pair  $i < j$  of distinct intervals has both indices in the left half  $[1..m]$ , both in the right half  $[m + 1..n]$ , or  $i \leq m < j$ . The two recursive calls handle the first two cases, so it suffices to show that CROSSOVERLAP handles the third. This is the hint with  $x = A[m + 1]$ : every interval in the left half has  $a_i < x$  and every interval in the right half has  $a_j \geq x$ .

Fix  $j > m$ . Because the array is sorted,  $a_i < a_j$  for every  $i \leq m$ , so  $\max\{a_i, a_j\} = a_j$  and

$$\text{overlap}(i, j) = \max\{0, \min\{b_i, b_j\} - a_j\}.$$

For fixed  $j$  this is a non-decreasing function of  $b_i$ , so the best partner for  $j$  in the left half is the interval with the largest right endpoint  $bmax$ , and the best cross overlap involving  $j$  is  $\max\{0, \min\{bmax, b_j\} - a_j\}$ . CROSSOVERLAP takes the maximum of this quantity over all  $j > m$ , and starting  $best$  at 0 supplies the outer  $\max\{0, \cdot\}$ . The base case is correct because a set of fewer than two intervals has no pair, and overlaps are never negative, so returning 0 never changes the maximum computed above it.

## Running time

CROSSOVERLAP makes one pass over each half, so it runs in  $O(n)$  time. The recursion splits the array into two halves and does  $O(n)$  extra work, so its running time obeys the mergesort recurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

ignoring floors and ceilings, which solves to  $T(n) = O(n \log n)$ . Adding the  $O(n \log n)$  preprocessing sort, the whole algorithm runs in

$O(n \log n)$  time.

**Sources and collaborators:** None.

**LLM use:** I used Claude (Anthropic) to draft this algorithm and explain its structure to me. I traced it by hand on the example in the handout and on several small inputs against the brute-force  $O(n^2)$  answer, and typeset the submission myself.